MAPÚA MALAYAN DIGITAL COLLEGE

A college under Mapúa Malayan Colleges Laguna

System Design Document of MotorPH Payroll System

Prepared and Presented by:

Torres, Jether Daniel A.
Bachelor of Science in information Technology
3rd Term - A.Y. 2024 - 2025

Intellectual Property Notice

This template is an exclusive property of **Mapua-Malayan Digital College** and is protected under **Republic Act No. 8293**, also known as the *Intellectual Property Code of the Philippines* (IP Code). It is provided solely for educational purposes within this course. Students may use this template to complete their tasks but may not **modify, distribute, sell, upload,** or **claim ownership** of the template itself. Such actions constitute copyright infringement under **Sections 172, 177, and 216** of the IP Code and may result in legal consequences. Unauthorized use beyond this course may result in legal or academic consequences.

Additionally, students must comply with the **Mapua-Malayan Digital College Student Handbook**, particularly with the following provisions:

- **Offenses Related to MMDC IT**:
    - **Section 6.2** - Unauthorized copying of files
    - **Section 6.8** - Extraction of protected, copyrighted, and/or confidential information by electronic means using MMDC IT infrastructure
- **Offenses Related to MMDC Admin, IT, and Operations**:
    - **Section 4.5** - Unauthorized collection or extraction of money, checks, or other instruments of monetary equivalent in connection with matters pertaining to MMDC

Violations of these policies may result in **disciplinary actions ranging from suspension to dismissal**, in accordance with the Student Handbook.

For permissions or inquiries, please contact MMDC-ISD at isd@mmdc.mcl.edu.ph.

TABLE OF CONTENTS

Section

1 **Introduction**
Purpose
Scope
Definitions Acronyms & Abbreviations
References

2 **Architectural Representation**
Use Case View
Use Case Diagram
Use Case Scenario
Logical View
System Users
Modules and Components

1. Introduction

MotorPH has relied on fragmented manual processes, such as paper timecards, spreadsheet-based payroll calculations, and branch-specific data reconciliation, to drive its growing workforce across multiple locations, which led to delayed payslips, calculation errors, and general bottlenecks that draw out the series of processes caused through repetition due to minor revisions in the payroll. The MotorPH Payroll System is designed to simplify and automate the once convoluted end-to-end payroll process for MotorPH, encompassing attendance tracking, salary computation, leave management, and compliance to government contributions. Driven by the shortcomings identified in the Business Requirement Specification (BRS) and the Current State Assessment of existing manual workflows, the system addresses inefficiencies, reduces processing times, and enhances data accuracy.

1.1. Purpose

This document provides a detailed description of how the MotorPH Payroll System and its components function and interact, serving as the basis and blueprint for the development, implementation, and maintenance of the said system.

1.2. Scope

The MotorPH Payroll System is designed to support the core administrative processes essential to compensating employees accurately, securely, and on time, primarily covering the digital management of employee records, attendance tracking through both remote and on-site channels, and the smoother handling of payroll computations, deductions, and reporting, whilst also accommodating for basic leave coordination and internal communication surrounding payroll-related matters, all within a secure, role-aware environment that respects data sensitivity and organisational hierarchy.

Nevertheless, despite the system being built to scale with the company's growing needs, it remains focused on the aforementioned detailed purposes, intentionally excluding non-payroll functionalities such as sales tracking, performance evaluation, or customer management. Likewise, it will not account for older historical records beyond a defined threshold nor will the project provision physical infrastructure for its integration. Its intent is not to replace broader management tools but rather to deliver a focused, dependable solution for the organisation's payroll ecosystem.

1.3. Definitions Acronyms and Abbreviations

Term	Definition
2FA	**Two-Factor Authentication -** a login method requiring two separate forms of identification.
BRS	**Business Requirement Specification -** a document outlining the business needs and expectations.
CRUD	**Create, Read, Update, Delete -** The four basic operations for data manipulation in databases and interfaces.
EID	**EmployeeID -** A unique identifier assigned to each employee in the MotorPH system.
HR	**Human Resources -** the department responsible for managing employee relations and data.
On/Offboarding	The structured process of accepting/exiting an employee from the organisation.
PDF	a file format that provides a digital image of text or text and graphics.
PTO	**Paid Time Off -** compensated leave granted to an employee.
QR Code	**Quick Response Code -** a machine-readable code used for on-site attendance validation.
RAD	**Requirements Analysis Document -** outlines system objectives, scope, and specifications.
RBAC	**Role-Based Access Control -** security system that limits user access based on their organisational role.
Ticketing	Earliest records of such a system being used is usually attributed to IBM. A ticket signified “your item (concern) is in the queue.” Internal request system used to track submissions and resolutions across HR, payroll, and admin.

1.4. References

Reference No.	Reference Document
1	Business Requirement Specification Document
2	Database Design Document
3	Requirements Analysis Document
4	Logic Design of Requirements
5	System Scope & Data Requirements

2. Architectural Representation

2.1. Use Case View

The MotorPH Payroll System is designed as a collection of modular services, each responsible for a distinct set of user goals. It is composed of the following modules and components:

2.1.1. Basic Employee Interactions Module

- Login: secure authentication with 2FA and session management
- Clock In/Out: off-site clock-in, QR-based on-site verification, clock-out confirmation and early-leave warning
- View Attendance: personal time-in/out logs, counts of present/absent/tardy days, overtime/undertime hours, leave usage
- Leave Calendar: bi-monthly team leave calendar and individual leave application status
- Salary Tracker: (mostly) precomputed earnings snapshot (hours, incentives, deductions, net pay) with timestamped estimates
- Tickets: submit and track requests (overtime, leave, timesheet updates, payroll disputes, tech support, general inquiries)

2.1.2. Team Management Module

- View Team Members' Info: roster with contact details and availability
- View/Edit Schedules: single-day, multi-day, and weekly pattern shift assignments
- Change Employee Timesheet: approve or edit attendance entries
- Tickets: process overtime/leave/timesheet/reschedule requests for team members

2.1.3. Team Management Module

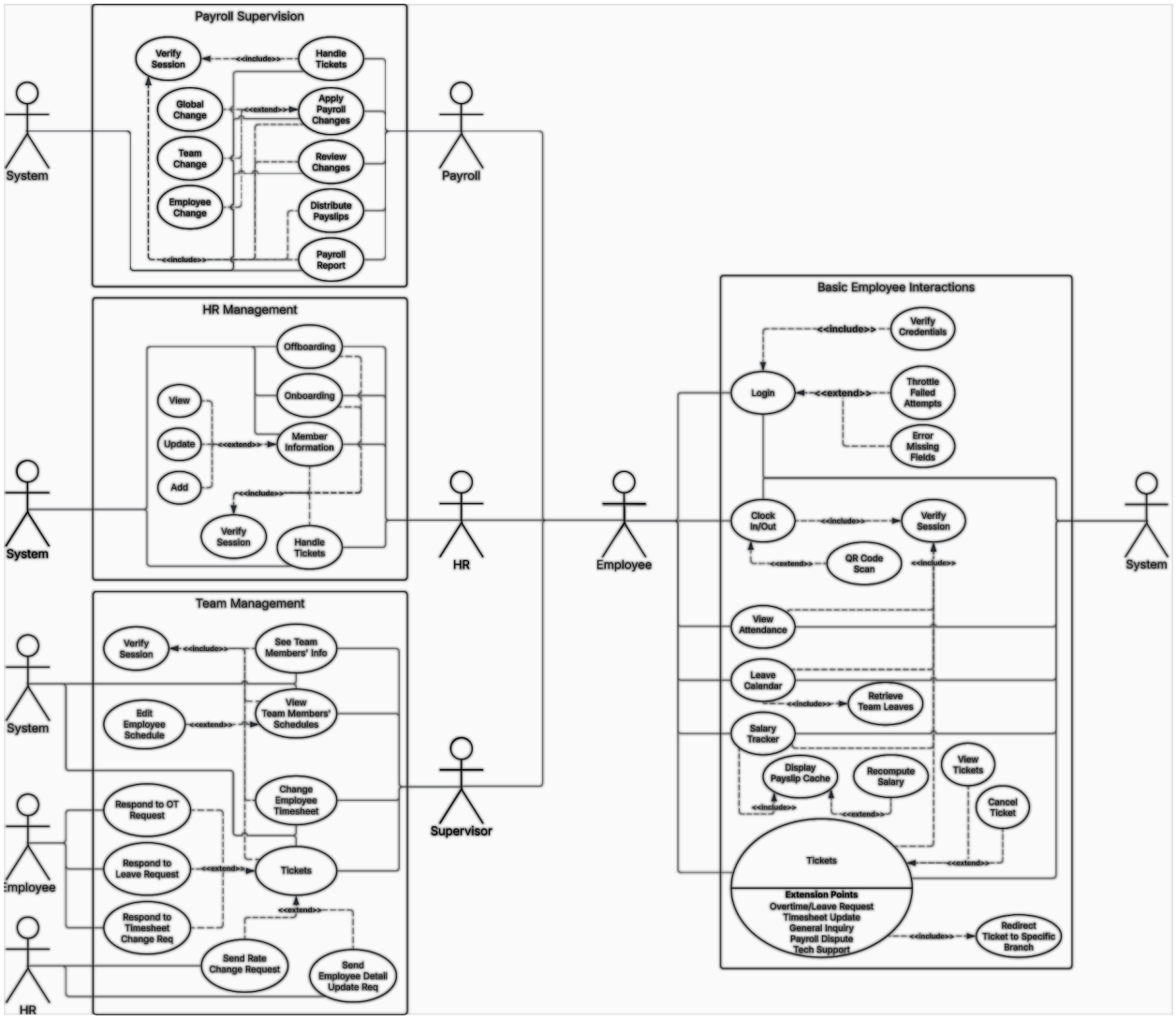
- Onboarding: add new hires with required personal and tax details
- Offboarding: process departures through clearance steps

- View/Update/Add Member Info: search and modify employee records, including mid-cycle rate changes
- Handle Tickets: resolve HR-direct and general inquiry tickets

2.1.4. Payroll Supervision Module

- Handle Tickets: review and audit payroll dispute or adjustment forms
- Apply Payroll Changes: individual, team, or global adjustments to incentives, rates, or deductibles
- Review Changes: audit trails and approval workflows
- Payroll Report: generate consolidated payroll summaries
- Distribute Payslips: secure generation and email delivery with retry logic

2.1.5. Use Case Diagram



2.1.6. Use Case Scenario

- **Basic Employee Interactions**
  An employee logs in, clocks in remotely, verifies on-site, checks today's attendance summary, views their salary estimate, and files an overtime request.
  1. **Secure Login**
     a. Employee opens the app and enters EmployeeID, Password, 2FA Code.
     b. System validates fields/format, checks password hash, verifies 2FA window, and applies progressive back-off on failures.
     c. On success, old sessions for that ID are terminated, a new session is created, and the employee is redirected to their Branch Dashboard.
  2. **Clock In On-Site**
     a. Employee taps Clock In: system confirms session, reads today's day type, prompts if it's a rest day/holiday/leave, then records off-site time and tags present or tardy.
     b. A QR code is displayed for 10 min.
     c. Upon arrival on-site, employee scans the QR: system validates it, updates status to "Onsite," recomputes tardy/present tag, and updates the attendance log.
  3. **View Today's Attendance**
     a. Employee opens Attendance tab: system fetches today's clock-in/out and displays status ("Present," "Tardy").
  4. **Salary Snapshot**
     a. Employee opens Salary Tracker: system retrieves their latest precomputed snapshot (hours, incentives, deductions, net pay) and shows "Estimate as of [timestamp]."
  5. **File Overtime Ticket**
     a. Employee selects Overtime Request, fills fromDate/toDate/reason, and submits.
     b. System generates a TicketID, queues it to the team lead, and shows the employee their ticket's status and position in queue.
- **Team Management**
  A team lead reviews the team roster, adjusts a member's weekly schedule, approves a leave request, and edits a timesheet entry.
  1. **View Roster & Availability**

    a. Lead logs in and opens Team Management: system validates session, fetches team members, today's DayType and attendance tags, and contact info.
    b. Roster UI shows each member's status (Present, OnLeave, Absent).
2. **Edit Weekly Schedule**
    a. Lead selects a member, chooses "Weekly Pattern," defines Mon-Fri 10:00-18:00, and clicks Save.
    b. System updates the member's schedule, logs the change, and notifies the member.
3. **Process Pending Tickets**
    a. Lead clicks into Pending Requests, sees a team member's Leave Request.
    b. Reviewing the form, the lead approves it; system updates the leave calendar and notifies the employee.
4. **Timesheet Correction**
    a. Lead opens the Timesheet Change ticket, sees requested clock times, and applies the correction to the attendance log.
    b. System marks the ticket Resolved, updates the log, and notifies the employee.

- **HR Management**
  An HR user onboards a new hire, updates an existing employee's branch, resolves an HR ticket, and processes a departure.
    1. **Onboarding**
        a. HR clicks Add New Hire, enters name, birthdate, branch, position, pay rate, tax IDs, then submits.
        b. System validates inputs, assigns EmployeeID, creates the record, and adds it to the Onboard Queue.
        c. After asset setup, HR marks the hire "Complete," removing them from the queue.
    2. **Record Update**
        a. HR searches an existing employee and changes their branch.
        b. System logs the change with timestamp and HRID and updates the employee record.
    3. **HR Ticket Handling**
        a. An employee submits a "General Inquiry" about benefits.

b. HR opens Tickets, reviews the form, replies in-system, marks it Resolved, and notifies the employee.

4. **Offboarding**
    a. HR clicks Process Departure for an exiting employee, enters departure date and clearance status, then submits.
    b. System adds them to the Offboard Queue; once final clearance is done, HR marks them Inactive and removes them from the queue.

- **Payroll Supervision**
  A payroll staff member applies a global tax-rate update, reviews an individual subsidy dispute, and then distributes final payslips.
    1. **Apply Global Adjustment**
        a. Staff opens Payroll Adjustments and chooses Global Scope.
        b. In the Adjustment Form, selects withholding tax rate > replace > 12%, then submits.
        c. System validates uniqueness, assigns dispute r equest ID, and sets status Pending.
    2. **Approve Individual Dispute**
        a. Employee disputed “RiceSubsidy” via ticket; staff opens an Individual Adjustment Form, adds rice subsidy > add > ₱200, and submits.
        b. System loads recorded values, applies the change to that employee’s payroll totals, logs the audit, and regenerates their payslip.
    3. **Distribute Payslips & Reports**
        a. After all approvals, staff clicks Distribute Payroll.
        b. System generates PDFs for each employee, emails them with retry logic for failures, and sends a collated report to the payroll team.
        c. A dashboard shows real-time status; on completion, the cycle is marked Distributed.
        d. An overall report summarising totals (gross, deductions, net by department or branch)

2.2. Logical View

2.2.1. System Users

- **RBAC Roles**

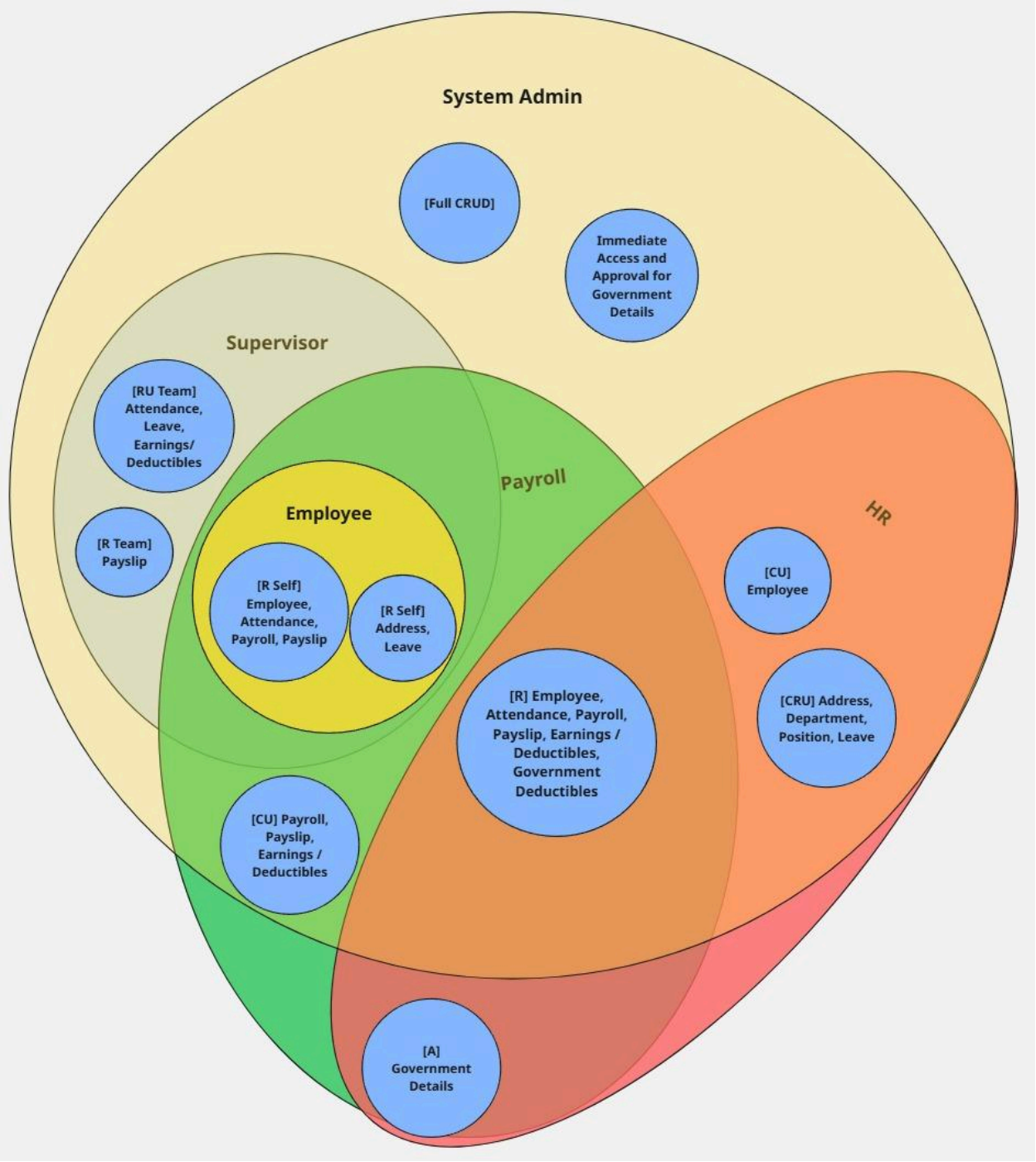
System Admin	Manages system configuration, user roles, security policies, and full database access.
HR	All lead functions + onboard/offboard management, handles employee records,

- **RBAC Permissions Matrix**

C: Create R: Read U: Update D: Delete A: Approval Req

Table/Entity	System Admin	HR	Payroll	Supervisor	Employee
Employee	C, R, U, D	C, R, U	R	R (Self)	R (Self)
Address	C, R, U, D	C, R, U	R (Self)	R (Self)	R (Self)
Department	C, R, U, D	C, R, U	-	-	-
Position	C, R, U, D	C, R, U	-	-	-
Leave	C, R, U, D	C, R, U	R (Self)	R, U (Team)	R (Self)
Attendance	C, R, U, D	R	R	R, U (Team)	R (Self)
Payroll	C, R, U, D	R	C, R, U	R (Self)	R (Self)
Payslip	C, R, U, D	R	C, R, U	R (Team)	R (Self)
Earnings/Deductibles	C, R, U, D	R	C, R, U	R, U (Team)	-
Government Deductibles	C, R, U, D	R	R	-	-
Government Details	C, R, U, D	A	A	-	-
Tax Table/Rates	C, R, U, D	-	-	-	-

- **Euler's Diagram of the Payroll System Access**



2.2.2. Modules and Components

- **Employee Login**
  Secure employee authentication with field validation, credential format checks, password hashing, multi-window 2FA verification, and progressive IP-based throttling and lockout.

Responsibility	Employees
Functionality	Authenticate credentials, enforce 2FA, throttle repeated failures, terminate old sessions, and redirect to branch dashboard.
User Interface	• Login page with fields for EmployeeID, Password, and 2FA code • Displays errors, lockout messages, and success redirects.
Interactions	1. User submits login form 2. Authentication Service validates fields, password hash, 2FA 3. Throttling service updates attempt counts (attempt restriction) 4. Session store creates session 5. Redirect to branch page
Dependencies	• Authentication API and credential store available • 2FA service reachable • Rate-limiter/cache service operating
Data Management	• `**EmployeeID**` (string) - Unique identifier for each employee. • `**Password**` (string) - represents the nominated password by the user during account registration. • `**Credentials**` (private map) - Container for EID and hashed passwords. • `**TwoFACodes**` (array[string]) - Array containing three valid 2FA codes: **2FA60s** (issued 60 s ago), **2FA30s** (30 s ago), **2FANow**.

	• `**FailedAttempts**` (int) - Counter of consecutive failures per IP/EID. • `**LastFailedTimestamp**` (datetime) - Timestamp of the most recent failed attempt for IP/EID. • `**CurrentTime**` (datetime) - System's current timestamp. • `**MissingFields**` (array) - Names of any login fields left blank.

- **Clock In / Out**
  Manage offsite/on-site clock-ins via QR codes and clock-out confirmations, tagging attendance accordingly.

Responsibility	Employees
Functionality	Handles time-in (offsite/onsite) and time-out actions, enforces day-type rules, and updates attendance tags.
User Interface	• "Clock In" and "Clock Out" buttons • QR code display • Confirmation dialogs
Interactions	1. Clock In click ○ Attendance Service logs offsite 2. QR generates ○ QR scan ○ Attendance Service updates onsite 3. Clock Out click 4. Confirmation dialogs 5. Finalise attendance log
Dependencies	• Active session (valid SessionID) • Calendar service for DayType and shift times

	• QR code generator service
Data Management	• `**EmployeeID**` (string) - unique identifier for the logged-in employee • `**SessionID**` (string) - token for the current authenticated session • `**ClockInTime**` (datetime) - recorded time when employee clocks in • `**ClockInStatus**` (enum: *Offsite* | *Onsite*) - indicates where the clock-in occurred • `**ClockOutTime**` (datetime) - recorded time when employee clocks out • `**DayType**` (enum: *Regular* | *RestDay* | *Holiday* | *Leave*) - type of day pulled from employee calendar • `**ShiftStartTime**` (time) - scheduled shift start for the employee on the current date • `**ShiftEndTime**` (time) - scheduled shift end for the employee on the current date • `**GracePeriod**` (duration) - allowable lateness before marking as “Tardy” • `**AttendanceTag**` (enum: *Present* | *Tardy* | *Absent*) - status of the day’s attendance • `**TotalWorked**` (duration) - total hours worked that day based on clock in/out • `**WorkHourTag**` (enum: *Regular* | *Overtime* | *Undertime*) - classification of worked hours • `**CurrentTime**` (timestamp) - current system time at action moment

- **Employee Attendance**
  Provides attendance metrics over a date range: logs, counts, and hour summaries for employees and their leads.

Responsibility	Employees and Leads
Functionality	Aggregates daily attendance entries into counts of present, absent, tardy, overtime, undertime, and leave usage.
User Interface	• Date-range picker • Displays daily log table and summary metrics
Interactions	1. User selects range 2. System retrieves logs and leaves 3. Displays report
Dependencies	• Attendance log store readable • Leave approval service available
Data Management	• `**EmployeeID**` (string) - unique identifier for the employee whose attendance is being viewed • `**SessionID**` (string) - token for the current authenticated session • `**DateRange**` (date, date) - start and end dates for the attendance report • `**AttendanceLog**` (record set) - list of clock-in/out entries retrieved from the database • `**AbsentCount**` (int) - number of days with no valid clock-in by threshold • `**PresentCount**` (int) - number of days with on-time clock-in • `**TardyCount**` (int) - number of days with late clock-in • `**OvertimeHours**` (duration) -

	total hours worked beyond scheduled time • `**UndertimeHours**` (duration) - total hours worked below scheduled time • `**LeaveTaken**` (int) - count of leave days used in the date range • `**LeaveBalance**` (int) - current remaining leave credits

- **Salary Tracker**
  Displays precomputed payroll projections (earnings, deductions, net) with scheduled and event-driven snapshot refresh.

Responsibility	Employees
Functionality	Serves cached salary snapshot and triggers recompute on schedule or relevant events.
User Interface	• Salary summary tab showing all components and “Estimate as of” timestamp.
Interactions	1. Tab open a. System fetches snapshot 2. On event/schedule a. Recomputation and snapshot refresh
Dependencies	• Salary snapshot cache available • Batch recompute service running • Event notifications (attendance, incentives, rate-changes)
Data Management	• `**EmployeeID**` (string) - unique identifier for the logged-in employee • `**SessionID**` (string) - token for the current authenticated session

	• `Position` (string) - employee's job title or role • `PayrollCycleStart` (date) - start date of the active payroll cycle • `PayrollCycleEnd` (date) - end date of the active payroll cycle • `HourlyWage` (currency) - base pay rate per hour • `RegHours` (duration) - total regular hours worked this cycle • `OTHours` (duration) - total overtime hours this cycle • `PTOHours` (duration) - total paid-time-off hours this cycle • `HolidayShifts` (int) - number of completed holiday shift days this cycle • `HolidayOTHours` (duration) - total holiday overtime hours this cycle • `TotalIncentives` (currency) - sum of all incentives awarded this cycle • `GrossPay` (currency) - precomputed earnings before deductions • `IncomeTaxRate` (percent) - current withholding tax rate • `IncomeTaxAmount` (currency) - precomputed income tax deduction • `OtherDeductibles` (currency) - sum of all other deductions this cycle • `NetPay` (currency) - precomputed take-home pay after deductions • `SnapshotTimestamp` (datetime) - when this snapshot was last computed

- **Tickets**
  Generic ticketing for OT, Leave, Timesheet, Dispute, Tech, Inquiry, and HR-direct, fully auditable and queue-driven.

Responsibility	Employees and Leads
Functionality	Creates, updates, cancels, and views tickets routed by type to handlers.
User Interface	• Six buttons/forms • “My Tickets” list with ticket details and cancel action
Interactions	1. Click form a. Fill b. Submit c. Queued 2. Handler updates status a. Notifies ticket sender 3. Employee views/cancels
Dependencies	• Ticket store service • Handler queue management
Data Management	• `**EmployeeID**` (string) - unique identifier for the requestor • `**SessionID**` (string) - token for the current authenticated session • `**TicketID**` (string) - unique reference code for each ticket • `**TicketType**` (enum) - one of: **Overtime**, **Leave**, **Timesheet**, **PayrollDispute**, **TechSupport**, **GeneralInquiry**, **HRDirect** • `**SubmitTime**` (datetime) - timestamp when ticket was created • `**ResolveTime**` (datetime) - timestamp when ticket was resolved

	• `Status` (enum) - one of: `Open`, `InProgress`, `Resolved`, `Cancelled` • `QueuePosition` (int) - ticket's position in the handler's queue • `HandlerID` (string) - identifier of the user or team assigned to process the ticket • `FormData` (map) - key/value pairs of the submitted form

- **Leave Tracker**
  Displays team roster availability and leave calendar for planning and coverage.

Responsibility	Employees
Functionality	Fetches and shows each teammate's status and upcoming leaves.
User Interface	• Team roster with contact and availability • Bi-monthly calendar view
Interactions	4. Click form a. Fill b. Submit c. Queued 5. Handler updates status a. Notifies ticket sender 6. Employee views/cancels
Dependencies	• Ticket store service • Handler queue management
Data Management	• `EmployeeID` (string) - unique identifier for the logged-in employee • `SessionID` (string) - token for the current authenticated session • `TeamMembers` (list of string) - identifiers of employees in the

	same team • `DayType` (enum) - one of: Regular, RestDay, Holiday, Leave, per team member and date • `Availability` (enum) - one of: Present, OnLeave, Absent, per team member and date • `CurrentDate` (date) - the date for which availability is displayed

- **Team Management**
  Allows leads to manage schedules, attendance edits, reschedules, and handle team-level tickets.

Responsibility	Supervisor
Functionality	Edit single/multi/weekly schedules; approve/reject OT/Leave/Timesheet/Reschedule requests.
User Interface	• Dashboard: roster + pending requests tabs • Schedule editor and ticket actions
Interactions	7. Load roster/logs a. Show status 8. Edit schedule a. Save+notify 9. Process tickets a. Apply changes or reject
Dependencies	• Attendance service for logs • Schedule store writable
Data Management	• `LeadID` (string) - unique identifier for the team lead (basically contains the same thing as `EmployeeID`)

	• `**SessionID**` (string) - token for the current authenticated session • `**TeamMembers**` (list\[string]) - EmployeeIDs of members reporting to this lead • `**MemberSchedule**` (map) - supports single-day, multi-day, and weekly shift patterns for each member • `**PendingTickets**` (list\[string]) - TicketIDs for OvertimeRequest, LeaveRequest, TimesheetChange, and RescheduleRequest routed to this lead • `**TicketStatus**` (map\[string]) - status of each TicketID (Open, InProgress, Resolved, Cancelled) • `**AttendanceLog**` (record set) - today's clock-in/out entries for each member

- **HR Handling**
  Enables HR to view stats, manage onboarding/offboarding, handle HR tickets, and edit employee records with audit.

Responsibility	HR
Functionality	Dashboard for company metrics, queues for hire/exit, ticket handling, and viewing and changing employee records.
User Interface	• HR dashboard: stats charts, hire/offboard queues, ticket list, record editor
Interactions	1. View stats 2. Add/edit employees 3. Process queues and tickets

Dependencies	• Employee data store • Audit log service • Rate-change form subsystem
Data Management	• `**LeadID**` (string) - unique identifier for the team lead (basically contains the same thing as **EmployeeID**) • `**SessionID**` (string) - token for the current authenticated session • `**TeamMembers**` (list\[string]) - EmployeeIDs of members reporting to this lead • `**MemberSchedule**` (map) - supports single-day, multi-day, and weekly shift patterns for each member • `**PendingTickets**` (list\[string]) - TicketIDs for OvertimeRequest, LeaveRequest, TimesheetChange, and RescheduleRequest routed to this lead • `**TicketStatus**` (map\[string]) - status of each TicketID (Open, InProgress, Resolved, Cancelled) • `**AttendanceLog**` (record set) - today's clock-in/out entries for each member

- **Payroll Supervision**
  Provides an adjustment form for payroll runs: allows scoped changes (individual/team/global) with audit and non-overlapping fields

Responsibility	Payroll Staff
Functionality	Build, review, approve or reject adjustment forms; apply and log

	changes to payroll data; regenerate payslips.
User Interface	• Adjustment form builder with dropdown fields, review panel, approve/reject buttons
Interactions	1. Approve/reject change a. Create form b. Add unique fields c. Submit d. System applies changes and logs
Dependencies	• Payroll run data store • Audit trail service • Payslip generation service
Data Management	• `**PayrollID**` (string) - unique identifier for the payroll run • `**SessionID**` (string) - token for the current authenticated session • `**DisputeRequestID**` (string) - unique identifier for each adjustment or dispute request • `**AffectedEmployeeID**` (string|null) - EmployeeID when targeting an individual; null for global/team adjustments • `**AdjustmentForm**` (list of records) - each record has: ○ `**FieldName**` (string) - the specific pay component to adjust (e.g. RiceSubsidy, WithholdingTax) ○ `**AdjustmentType**` (enum: *Add|Subtract|Replace*) - operation to perform on the original value ○ `**Amount**` (currency or

	percent) - magnitude of the change • `GroupScope` (enum: *Individual|Team|Global*) - defines whether the adjustment applies to one employee, a team, or all employees • `DisputeStatus` (enum: *Pending|Approved|Rejected*) • `RecordedValues` (map) - snapshot of original computed fields for each affected EmployeeID before any adjustment

- **Payroll Distribution**
  Automates secure generation and email delivery of payslips and summary reports, with retry and status tracking.

Responsibility	Payroll Staff
Functionality	Generates PDF payslips and aggregate report, emails to employees and staff, retries failures, and logs delivery.
User Interface	• Distribution dashboard showing real-time status grid with retry controls
Interactions	1. Click “Distribute” 2. Generate files 3. Send emails 4. Update statuses
Dependencies	• Payslip/report generation service • Email SMTP/service with retry support • Employee email directory
Data Management	• `PayrollID` (string) - unique identifier for the payroll cycle

	• `**SessionID**` (string) - token for the current authenticated session • `**PayslipList**` (list\[file]) - individual payslip PDF files for each employee • `**PayrollReport**` (file) - aggregate payroll report (e.g. CSV or PDF) for payroll staff • `**EmployeeEmails**` (map\[string]) - mapping of EmployeeID to email address • `**DistributionStatus**` (map\[string]) - status of each payslip email (Pending, Sent, Failed)